TASK 4 REPORT SECTION

4. Deployment

4.1 Deployment Strategy

The deployment strategy translates the validated Random Forest classifier from Task 2 into a usable decision-support component for the SDHSA project.

Intended users

Provincial and local government planning officials.

NGOs and community outreach coordinators.

Social development analysts who prioritise household support.

Decisions supported

The component supports screening and prioritisation, not eligibility decisions. It produces a predicted vulnerability flag and probability for each household so that users can:

Prioritise households for further assessment or outreach.

Rank a list of households by predicted vulnerability probability.

Compare predicted vulnerability across provinces, geo-types and household sizes (aggregate level only).

It does not decide whether a household receives assistance — that remains a human decision, consistent with Task 3.

nputs

A CSV file containing the same feature columns used in the final model after leakage removal (see Task 2). These include:

Geographic: province, province_code, geo_type_code, metro_code, survey-year indicators.

Dwelling: main_dwelling_code, wall_material_code, roof_material_code, floor_material_code, wall_condition_code, roof_condition_code, floor_condition_code, total_rooms, tenure_code, rent_raw, rdp_housing_code, housing_subsidy_code.

Household composition: household_size, hh_size_person, n_children, n_elderly, hh_head_education, dependency_ratio, hh_size, head_age, head_sex_code, head_population_group_code.

Energy: cooking_energy_code, lighting_energy_code, space_heating_energy_code, loadshedding_frequency_code, alternative_lighting_code, alternative_cooking_code.

Resources: expenditure_band_code, total_vehicles.

Food-stress day counts: skipped_meal_days, ate_less_days, ran_out_days.

The target column vulnerable must not be present in the uploaded file.

Outputs

.pred_class — Vulnerable / Not vulnerable.

.pred_Vulnerable — probability between 0 and 1.

A downloadable CSV of scored households.

Validation messages for malformed input.

Upload CSV → Schema validation → Range/type checks → Apply saved recipe (bake)

→ Random Forest predict → Results table + probability → Download CSV

→ Log run (timestamp, rows, model version)

All scoring uses the saved final_model.rds and final_recipe.rds from Task 2. The recipe is only baked, never re-fitted, which preserves the leakage-free design and guarantees identical preprocessing between training and deployment.

4.2 Deployment Tool Evaluation

Tool,Strengths,Weaknesses,Fit for SDHSA

R/ShinyNative to the existing R pipeline; no model rewrite; free; interactive; supports CSV upload/download and validation messages

Requires R runtime and a host (e.g., shinyapps.io, Posit Connect) Recommended

R Plumber (REST API) Reusable by other systems; clean separation of model and UI No interface for analysts; needs separate front-end Useful future integration, not for this milestone

Quarto / R Markdown report Reproducible, easy to produce Not interactive; no live scoring of new files Good for static reporting only

Power BI / Tableau Strong visualisation Model would need export or re-implementation → risk of pipeline drift and leakage Not recommended

Recommendation: R/Shiny. The model, recipe, leakage-removal logic, and validation design already live in R. Deploying in Shiny keeps one codebase, avoids re-implementing preprocessing, and directly supports the Task 3 condition that the tool is used with human oversight.

4.3 Deployed Analytical Component

The Shiny app contains:

Upload panel — CSV upload and a "Validate & Score" button.

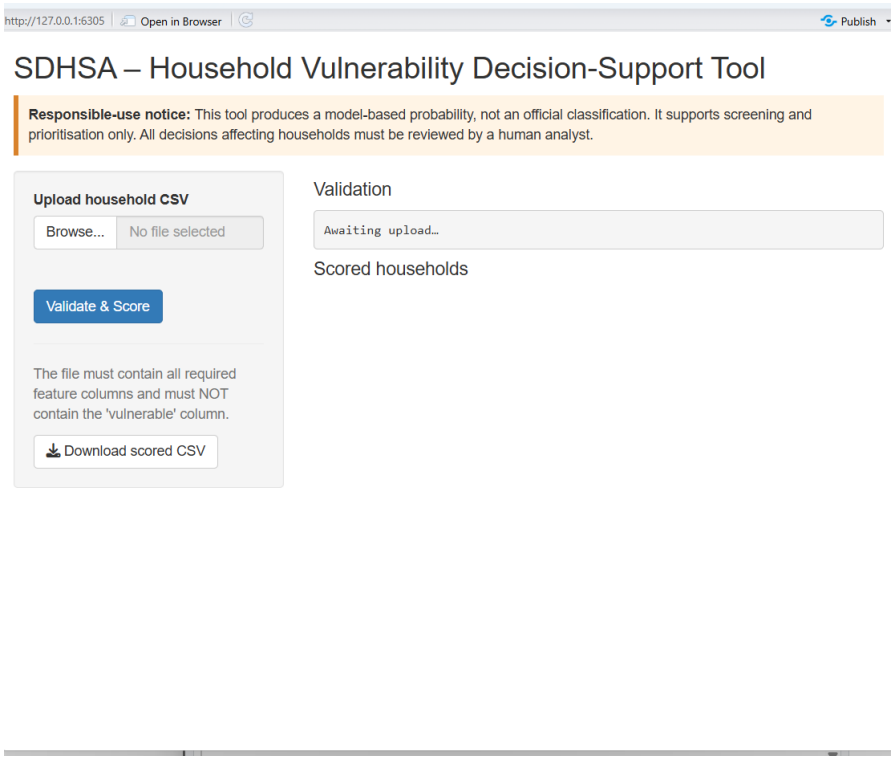
Validation output — missing columns, wrong types, out-of-range warnings.

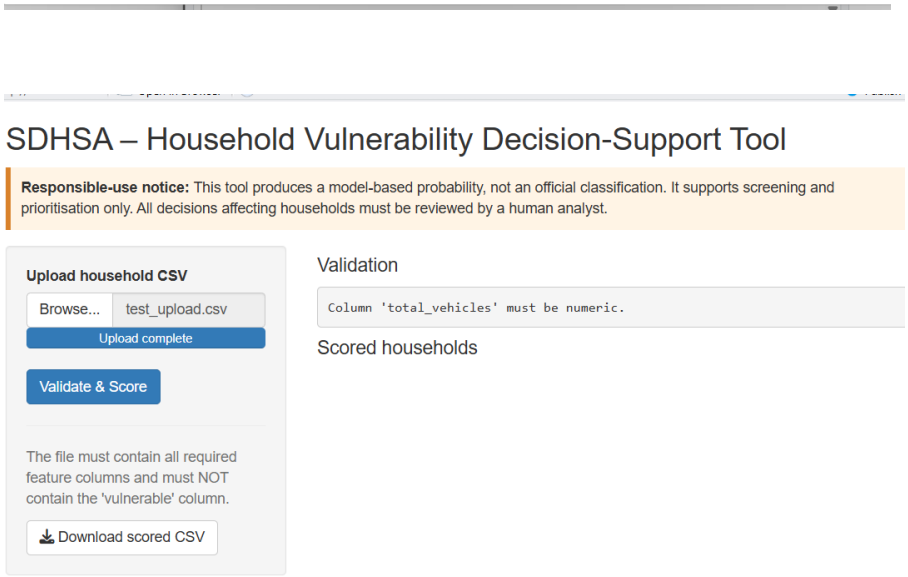
Results table — household rows with .pred_class and .pred_Vulnerable

Download button — scored CSV for downstream use.

Responsible-use notice — a permanent banner reminding users the output is a model-based proxy and must be reviewed by a human.

Household identifiers are not displayed publicly; only an internal row index is shown.





4.4 New-Data Validation and Scoring

Schema check — every required feature column must be present; the vulnerable column must be absent.

Type check — numeric and categorical columns must match the training types.

Range check — numeric values compared against training min/max; out-of-range values are flagged but not silently dropped.

Missing-value handling — the saved recipe applies median imputation for numeric and mode imputation for nominal, exactly as in training.

Encoding — step_dummy() applies the same dummy coding learned in training (unseen levels are handled by the recipe).

Scoring — predict(rf_fit, new_data = baked) returns class and probability.

Logging — timestamp, file name, row count, model version, and (optionally) a hash of the input file are appended to a log.

4.5 Monitoring Strategy and Measures

Element and Specification

Primary performance metric F1-score for the Vulnerable class (matches Task 2 selection rationale)

Secondary metrics Recall, Precision, Balanced Accuracy, ROC-AUC

Baseline (from Task 2 test set) F1 ≈ 0.89, Recall ≈ 0.89, ROC-AUC ≈ 0.93

Alert threshold F1 drops more than 10% below baseline (i.e., F1 < 0.80) when labelled feedback becomes available

Data drift indicators Population Stability Index (PSI) on key numeric features; chi-square on key categorical features; KS test on predicted probabilities

Prediction drift Shift in the proportion of Vulnerable predictions compared with the training distribution (≈ 64% vulnerable)

Fairness monitoring Per-group F1 / Recall by province, geo-type and household-size band. Flag any group whose Recall is more than 10 percentage points below the overall figure.

Retraining triggers (a) F1 below threshold, (b) PSI > 0.25 on three or more features, (c) a new GHS survey wave becomes available, (d) material fairness gap detected

Retirement conditions Business objective changes; target definition changes; model repeatedly fails fairness review; a materially better model is validated

Monitoring is manual but documented: a quarterly review meeting compares the current dashboard log against the baseline and flags any trigger.

4.6 Reproducible Repository

text

/sdhsa-milestone3

```
├── data/
│   ├── raw/              # original GHS files (not committed if sensitive)
│   ├── interim/
│   └── processed/        # final analytical dataset
├── R/
│   ├── 01_prep.R         # Milestone 2 preprocessing
│   ├── 02_model.R        # Task 1/2 modelling
│   └── 03_eval.R         # Task 2 evaluation
├── artefacts/
│   ├── final_model.rds     # saved Random Forest workflow
│   ├── final_recipe.rds    # saved preprocessing recipe
│   ├── feature_meta.rds    # column names, types, ranges
│   └── model_comparison_matrix.csv
├── app/
│   └── app.R             # Shiny deployment
├── reports/
│   ├── Milestone3_Task1.docx
│   ├── Milestone3_Task2.html
│   ├── Task_3_Business_Ethical_Evaluation.pdf
│   └── Milestone3_Task4_Deployment.pdf
├── monitoring/
```

│ └── monitoring_log.csv

├── ethics_ai_log.md

└── README.md

README.md documents how to run 01_prep.R → 02_model.R → 03_eval.R → app/app.R in order, with set.seed(381) preserved.